Technische Datenspezifikation des Verzeichnisdienstes

Version 0.9

Inhaltsverzeichnis

1 Einleitung 4
1.1 Ziel und Geltungsbereich 4
1.2 Normative Einordnung 4
1.3 Schlüsselwörter 4
2 Normative Release-Artefakte 5
3 Verbindliche Festlegungen und Datenhoheit 6
3.1 Datenhoheit 6
3.2 Berechtigung 8
3.3 Umgebungstrennung 8
4 Datenmodell des DirectoryRecord 9
4.1 Schlüssel, Revision und ETag 9
4.2 Eigenschaften des vollständigen DirectoryRecord 9
4.3 Kommunikationsparameter 10
4.4 Kontakte und Gültigkeit 11
4.5 Lifecycle 11
4.6 Verfügbarkeitsüberwachung 11
4.7 Manueller Override und geplante Wartung 11
4.7.1 Ermittlung des wirksamen Betriebsstatus 12
4.7.2 Setzen, Ersetzen und Aufheben 12
4.7.3 Zeitgesteuerte Wartung 13
4.8 Initialzustand, Erweiterbarkeit und Größe 13
5 Synchrone Verzeichnisdienst-API 15
5.1 Ressource und Operationen 15
5.2 PUT-Objekt 15
5.3 PATCH-Objekt 16
5.4 Löschung 16
5.5 Signatur- und Response-Regeln 16
5.6 ETag, Retry, Idempotenz und atomare Verarbeitung 17
5.7 Betriebs- und Retryprofil 18
6 Zustandsänderungen und WebSocket-Ereignisse 19
7 WebSocket-Integration und externe Schemata 20
7.1 Schemaauslagerung 20
7.2 AsyncAPI-Vertrag 20
8 Serverseitige Semantik außerhalb von JSON Schema 21
9 Beispiele 22
9.1 DirectoryRecordWrite 22
9.2 Geplante Wartung setzen 22

9.3 Vollständiger DirectoryRecord während aktiver Wartung 23
9.4 Löschbestätigung 24

10 Konformität und Veröffentlichung 25
11 Quellen 26

1 Einleitung

1.1 Ziel und Geltungsbereich

Dieses Dokument ist die technische Datenspezifikation des zentralen Verzeichnisdienstes am Data Hub. Es konkretisiert den vollständigen DirectoryRecord, die synchrone Pflege- und Abfrage-API sowie die Zuordnung der externen WebSocket-Nachrichtenschemata. Arbeitsgrundlage sind die API-Guideline Wasserstoffmarkt, Stand 11.07.2026 [API-GL], die aktuelle JWS-Spezifikation [JWS-SPEC] und die WebSocket-Spezifikation, Stand 12.07.2026 [WS-SPEC].

Die Spezifikation umfasst den vollständigen Verzeichniseintrag, die marktpartnerseitigen Schreibobjekte, den manuellen Betriebsstatus-Override einschließlich geplanter Wartungen, die Löschbestätigung, das synchrone OpenAPI-Dokument und das asynchrone AsyncAPI-Dokument.

MUST Der vollständige DirectoryRecord MUSS in synchronen Responses und in notification.payload.entry identisch verwendet werden.

MUST NOT In Version 1 DARF eine eigenständige, dauerhaft gespeicherte Signatur des DirectoryRecord NICHT verwendet werden. Die Signaturpflicht für HTTP-Requests, fachlich relevante HTTP-Responses und signaturpflichtige WebSocket-Client-Nachrichten bleibt unverändert bestehen.

1.2 Normative Einordnung

Die API-Guideline ist das übergeordnete Regelwerk. [JWS-SPEC] ist für Algorithmen, JWS-Serialisierung, Signaturrepräsentation, Zertifikate, Zeitfenster und Replay-Schutz maßgeblich. [WS-SPEC] ist für WebSocket-Verbindungsaufbau, Nachrichtenabläufe, Zustandslogik, Fehler- und Close-Codes sowie Betriebsparameter maßgeblich. Dieses Dokument ist für Datenmodell, Datenhoheit, synchrone DirectoryRecord-Operationen und die Zuordnung der maschinenlesbaren Artefakte maßgeblich.

Für formale Struktur- und Wertebereichsprüfungen sind die JSON-Schemata maßgeblich. OpenAPI und AsyncAPI bilden den HTTP- beziehungsweise WebSocket-Vertrag maschinenlesbar ab. Abweichungen zwischen den Artefakten sind ein Releasefehler und begründen kein Wahlrecht der Implementierung.

1.3 Schlüsselwörter

Die Schlüsselwörter MUSS/MÜSSEN, DARF NICHT/DÜRFEN NICHT, SOLLTE/SOLLTEN und KANN/KÖNNEN sind normativ im Sinne von RFC 2119 und RFC 8174 zu verstehen, wenn sie in Großbuchstaben verwendet werden.

2 Normative Release-Artefakte

Datei	Version / kanonische ID	Zweck
directory-record.schema.yaml	urn:h2:directory:record:v1	DirectoryRecord, DirectoryRecordWrite, ManualOverrideRequest und DirectoryRecordDeletionResult
ws-client-message.schema.yaml	urn:h2:directory:ws:client-message:v1	Geschlossene signaturpflichtige WebSocket-Client-Nachrichten
ws-server-message.schema.yaml	urn:h2:directory:ws:server-message:v1	Erweiterbare WebSocket-Server-Nachrichten und DirectoryRecord-Referenz
openapi-directory-service-v1.yaml	OpenAPI 3.1.2 / 1.0.0	Synchrone GET-, PUT-, PATCH- und DELETE-Operationen
asyncapi-directory-websocket-v1.yaml	AsyncAPI 3.0.0 / 1.0.0	WebSocket-Kanal, Nachrichten, Upgrade- und Betriebsparameter

MUST Die fünf maschinenlesbaren Dateien MÜSSEN gemeinsam mit dieser Datenspezifikation versioniert und veröffentlicht werden. Nicht zueinander passende Einzelstände DÜRFEN NICHT kombiniert werden.

MUST URN-Referenzen MÜSSEN gegen das freigegebene lokale oder kontrolliert bereitgestellte Artefaktbündel aufgelöst werden.

MUST NOT Implementierungen DÜRFEN referenzierte Schemata NICHT zur Laufzeit aus einer unkontrollierten Netzquelle nachladen.

3 Verbindliche Festlegungen und Datenhoheit

Die folgenden Festlegungen bilden die verbindliche Grundlage für den DirectoryRecord und seine Pflege. Sie konkretisieren Signaturführung, Datenhoheit, Zieladressierung, Capabilities, Betriebsstatus und Löschverhalten für Version 1.

MUST NOT Der DirectoryRecord DARF in Version 1 KEINE dauerhaft gespeicherte Record-Signatur und kein Zertifikat enthalten. Nachrichten werden gemäß [JWS-SPEC] signiert.

MUST Die Datenhoheit MUSS getrennt behandelt werden. Katalog- und Lifecycle-Daten liegen bei der API-Governance; allowedRoles und capabilities werden vom Marktpartner als zugelassene Teilmengen gepflegt; Statusdaten werden durch den Verzeichnisdienst geführt.

MUST Für communicationParameters MUSS genau eine Zieladresse verwendet werden. targetUrl MUSS eine absolute HTTPS-URL ohne Query, Fragment, Benutzerinformation, IP-Literal oder abschließenden Schrägstrich sein.

MUST capabilities MUSS als eindeutige Liste registrierter lowerCamelCase-Codes geführt werden; ein leeres Array ist zulässig.

MUST Manuelle Overrides MÜSSEN über PATCH auf derselben Record-Ressource gesetzt, ersetzt oder aufgehoben werden. action=set beziehungsweise action=clear; geplante Wartungen werden zeitgesteuert aktiviert.

MUST Neue und wiederangelegte Einträge MÜSSEN mit down / initialRegistration beginnen.

MUST Eine erfolgreiche Löschung MUSS 200 OK mit DirectoryRecordDeletionResult liefern. 204 No Content wird nicht verwendet.

3.1 Datenhoheit

Die Datenhoheit ist nach Datenbereichen getrennt. Verzeichnisschlüssel, Governance-Daten, marktpartnerseitig pflegbare Angaben und systemseitige Statusdaten besitzen jeweils einen festgelegten Änderungsweg.

MUST providerId, apiId und majorVersion werden über Pfad, Authentisierung und API-Governance bestimmt und DÜRFEN NICHT als Body-Daten geändert werden.

MUST revision wird ausschließlich durch den Verzeichnisdienst und ausschließlich atomar fortgeschrieben.

MUST serviceName, serviceDescription, lifecycleStatus, deprecatedSince, sunsetAt und replacement unterliegen der API-Governance und werden über den internen Governance-Prozess gepflegt.

MUST allowedRoles, communicationParameters, capabilities, contacts, validity und availability-Monitoring werden durch den verantwortlichen Marktpartner über PUT mit If-None-Match beziehungsweise If-Match gepflegt.

MUST operationalStatus und die zugehörigen Metadaten werden durch den Verzeichnisdienst aus Lifecycle, Override und WebSocket-Monitoring abgeleitet.

MUST manualOverride wird durch den Verzeichnisdienst geführt und kann durch einen berechtigten Marktpartner oder die Governance über PATCH mit If-Match ausgelöst werden.

MUST availabilityMonitoringStatus wird durch den Verzeichnisdienst ausschließlich aus der WebSocket-Zustandslogik geführt.

Die folgende Tabelle fasst die Datenhoheit und den jeweiligen Änderungsweg als Kurzreferenz zusammen:

Datenbereich	Eigenschaften	Datenhoheit	Öffentlicher Änderungsweg
Verzeichnisschlüssel	providerId, apiId, majorVersion	Pfad, Authentisierung, API-Governance	Nicht als Body-Daten änderbar
Revision	revision	Verzeichnisdienst	Ausschließlich atomar durch den Dienst
API-Katalog	serviceName, serviceDescription, lifecycleStatus, deprecatedSince, sunsetAt, replacement	API-Governance	Interner Governance-Prozess
Marktpartnerdaten	allowedRoles, communicationParameters, capabilities, contacts, validity, availabilityMonitoring	Verantwortlicher Marktpartner	PUT mit If-None-Match oder If-Match
Betriebsstatus	operationalStatus und zugehörige Metadaten	Verzeichnisdienst	Ableitung aus Lifecycle, Override und WebSocket-Monitoring
Manual Override	manualOverride	Verzeichnisdienst; ausgelöst durch berechtigten Marktpartner oder Governance	PATCH mit If-Match
Monitoringstatus	availabilityMonitoringStatus	Verzeichnisdienst	Nur WebSocket-Zustandslogik

3.2 Berechtigung

MUST Jeder erfolgreich authentisierte EMT.API-Marktpartner darf jeden vorhandenen Directory-Record über GET lesen. allowedRoles steuert die Nutzung der beschriebenen fachlichen API, nicht die Sichtbarkeit des Verzeichniseintrags.

MUST PUT, PATCH und DELETE DÜRFEN durch einen Marktpartner ausschließlich für die eigene providerId ausgeführt werden. serviceName, serviceDescription und Lifecycle-Daten werden nicht über öffentliche OpenAPI-Pfade gepflegt.

Interne Governance-Änderungen unterliegen denselben Regeln für Revision, Audit, atomaren Commit und Transactional Outbox wie öffentliche Pflegeoperationen.

3.3 Umgebungstrennung

MUST PROD und TEST MÜSSEN getrennte DirectoryRecord-Bestände führen. Verzeichnisschlüssel, Revisionen, Revisions-Hochwassermarken, Monitoringzustände, Idempotenzdaten, Auditdaten und Outbox-Ereignisse sind je Umgebung unabhängig.

MUST Der DirectoryRecord DARF kein Umgebungsfeld enthalten. Die Umgebung ergibt sich ausschließlich aus der adressierten Verzeichnisdienstinstanz. targetUrl MUSS einen Endpunkt derselben Umgebung bezeichnen; eine umgebungsübergreifende Verarbeitung ist unzulässig.

4 Datenmodell des DirectoryRecord

4.1 Schlüssel, Revision und ETag

`<providerId>/<apiId>/<majorVersion>`

MUST providerId MUSS eine 13-stellige numerische MP-ID, apiId eine API-Kennung nach dem Muster ^H2API[0-9]{5}$ und majorVersion eine positive Ganzzahl ohne Präfix v sein.

MUST revision MUSS je Verzeichnisschlüssel streng monoton steigen. Bei der erstmaligen Anlage beträgt sie 1; nach einer Löschung bleibt die Revisions-Hochwassermarke erhalten und eine Wiederanlage setzt sie um genau 1 fort.

MUST Der starke HTTP-Entity-Tag MUSS exakt "rev-<revision>" lauten. ETag, DirectoryRecord.revision und die Revision einer zugehörigen WebSocket-Benachrichtigung MÜSSEN denselben Stand bezeichnen.

4.2 Eigenschaften des vollständigen DirectoryRecord

Eigenschaft	Typ	Pflicht	Datenhoheit	Regel
providerId	string	ja	Schlüssel	13 Ziffern
apiId	string	ja	Governance	^H2API[0-9]{5}$
majorVersion	integer	ja	Governance	1 bis 2.147.483.647
revision	integer	ja	System	1 bis 9.007.199.254.740.991
serviceName	string	ja	Governance	1 bis 256 Zeichen
serviceDescription	string	nein	Governance	1 bis 1.024 Zeichen
allowedRoles	array	ja	Marktpartner	Nichtleer, eindeutig, katalogseitige Teilmenge
communicationParameters	object	ja	Marktpartner	Genau eine Zieladresse, Protokoll, Methoden
capabilities	array	ja	Marktpartner	Eindeutige registrierte Codes; leer zulässig
contacts	object	ja	Marktpartner	Technischer Kontakt Pflicht, fachlicher optional
validity	object	ja	Marktpartner	validFrom Pflicht; validUntil optional
lifecycleStatus	string	ja	Governance	experimental, beta, active, deprecated, sunset, retired
deprecatedSince	date-time	bedingt	Governance	Pflicht bei deprecated
sunsetAt	date-time	bedingt	Governance	Pflicht bei deprecated und sunset

Eigenschaft	Typ	Pflicht	Datenhoheit	Regel
replacement	object	bedingt	Governance	Pflicht bei deprecated: successor oder none
operationalStatus	string	ja	System	operational, degraded, maintenance, down
operationalStatusSource	string	ja	System	lowerCamelCase-Ursprungscode
operationalStatusChangedAt	date-time	ja	System	Beginn des aktuell wirksamen Zustands
operationalStatusReason	string	ja	System	lowerCamelCase-Grundcode
operationalStatusDescription	string	bedingt	System	Pflicht bei degraded, maintenance und down sowie aktivem Override
operationalStatusExpectedEnd	date-time	bedingt	System	Pflicht bei aktiver maintenance
operationalStatusReference	string	nein	System	Wartungs- oder Störungsreferenz
manualOverride	object	bedingt	System	scheduled oder active
availabilityMonitoring	object	ja	Marktpartner	enabled und websocketHeartbeat
availabilityMonitoringStatus	object	ja	System	inactive, confirmed, unconfirmed, monitoringUnavailable

4.3 Kommunikationsparameter

Eigenschaft	Pflicht	Regel
targetUrl	ja	Absolute HTTPS-URI, höchstens 2.048 Zeichen; DNS-Name; keine Benutzerinformation, IP-Literale, Query, Fragment oder abschließender Schrägstrich.
protocol	ja	Fester Wert https.
supportedHttpMethods	ja	Nichtleere eindeutige Liste aus GET, POST, PUT, PATCH und DELETE.

MUST Der Verzeichnisdienst MUSS targetUrl zusätzlich gegen den für apiId und majorVersion zugelassenen Nutzungskontext prüfen. Ein syntaktisch gültiger, fachlich aber unzulässiger Endpunkt wird mit 422 Unprocessable Content abgelehnt.

4.4 Kontakte und Gültigkeit

contacts.technical ist verpflichtend, contacts.business optional. Jeder Kontakt enthält name und mindestens email oder phone; organisation und serviceHours sind optional. Nicht befüllte optionale Eigenschaften werden weggelassen, null ist nicht zulässig.

MUST validity.validUntil MUSS, wenn vorhanden, nach validity.validFrom liegen. Vor validFrom und ab validUntil DARF der Eintrag nicht für neue Routing-, Weiterleitungs-, Zustell- oder Bereitstellungsentscheidungen verwendet werden.

4.5 Lifecycle

MUST Bei lifecycleStatus=deprecated MÜSSEN deprecatedSince, sunsetAt und replacement vorhanden sein. replacement.kind=successor enthält apiId und majorVersion des Nachfolgers; replacement.kind=none kennzeichnet eine ersatzlose Abkündigung.

MUST Bei lifecycleStatus=sunset MUSS sunsetAt vorhanden sein. deprecatedSince MUSS vor sunsetAt liegen; bei erstmaliger Setzung muss sunsetAt in der Zukunft liegen.

MUST Bei lifecycleStatus=retired MÜSSEN operationalStatus=down, operationalStatusSource=lifecycleRetired, operationalStatusReason=lifecycleRetired und availabilityMonitoringStatus.state=inactive mit reason=lifecycleRetired gelten. Ein Manual Override darf dann nicht vorhanden sein.

4.6 Verfügbarkeitsüberwachung

```
"availabilityMonitoring": {
  "enabled": true,
  "mechanism": "websocketHeartbeat"
}
```

Bei enabled=false gilt außerhalb retired availabilityMonitoringStatus.state=inactive mit reason=monitoringDisabled. Bei enabled=true ohne erstmalig bestätigte availabilityStart-Nachricht gilt unconfirmed/monitoringEnabledAwaitingStart; allein hieraus beginnt keine Down-Karenz.

confirmed, unconfirmed, monitoringUnavailable, Heartbeat-Timeout, Karenzzeit und Mass-Loss-Circuit-Breaker richten sich nach [WS-SPEC]. Eine Änderung von availabilityMonitoringStatus.state erhöht die Revision; ein Heartbeat im bereits bestehenden Zustand confirmed nicht.

4.7 Manueller Override und geplante Wartung

Eigenschaft	Pflicht	Bedeutung
state	ja	scheduled bei einer zukünftigen geplanten Wartung; active bei aktuell wirksamem Override.

Eigenschaft	Pflicht	Bedeutung
operationalStatus	ja	Gewünschter wirksamer Status; scheduled ist nur für maintenance zulässig.
reason	ja	Maschinenlesbarer lowerCamelCase-Grund.
description	ja	Beschreibung der Ursache oder Auswirkung.
reference	nein	Wartungs- oder Störungsreferenz.
setAt	ja	Serverseitiger Commit-Zeitpunkt des set-Requests.
setBy	ja	Serverseitig aus Authentisierung und Rolle gebildeter Akteur.
activatedAt	bei active	Zeitpunkt der tatsächlichen Aktivierung.
validUntil	bei maintenance	Ende des Overrides; bei maintenance identisch zu maintenanceWindow.end.
maintenanceWindow	bei maintenance	start und end; start liegt vor end.

4.7.1 Ermittlung des wirksamen Betriebsstatus

MUST Der wirksame Betriebsstatus MUSS in folgender Priorität bestimmt werden: lifecycleStatus=retired erzwingt down; andernfalls bestimmt ein aktiver manualOverride den Status; andernfalls gilt der ohne aktiven Override maßgebliche Initial- oder Monitoringstatus.

MUST Ein manualOverride mit state=scheduled DARF den wirksamen operationalStatus und dessen Metadaten noch nicht bestimmen. Die Implementierung MUSS den ohne aktiven Override maßgeblichen Grundzustand und seine Metadaten deterministisch fortführen oder rekonstruieren können; hierfür ist kein zusätzliches öffentliches DirectoryRecord-Feld vorgeschrieben.

MUST Während eines aktiven Overrides MUSS availabilityMonitoringStatus vollständig weitergeführt werden. Automatische Monitoringereignisse DÜRFEN den wirksamen operationalStatus nicht verändern, MÜSSEN aber bei Aufhebung oder Ablauf des Overrides in die sofortige Neubewertung des Grundzustands eingehen.

MUST operationalStatus=maintenance und operationalStatus=degraded DÜRFEN in Version 1 nur durch einen aktiven manualOverride entstehen. Bei active MÜSSEN operationalStatus, operationalStatusSource, operationalStatusReason und operationalStatusDescription den Override-Werten entsprechen; operationalStatusChangedAt MUSS activatedAt entsprechen. Soweit validUntil oder reference vorhanden sind, MÜSSEN operationalStatusExpectedEnd beziehungsweise operationalStatusReference den entsprechenden Override-Werten entsprechen.

4.7.2 Setzen, Ersetzen und Aufheben

MUST Ein PATCH mit action=set MUSS einen vorhandenen geplanten oder aktiven Override atomar ersetzen. setAt und setBy werden durch den Verzeichnisdienst gebildet und dürfen nicht im Request enthalten sein.

MUST Liegt bei Annahme einer Wartung maintenanceWindow.start nach der Serverzeit, MUSS der Override mit state=scheduled gespeichert werden. Liegt start nicht nach der Serverzeit, aber

vor end, MUSS der Override unmittelbar mit state=active und activatedAt=Commit-Zeitpunkt gespeichert werden. Liegt end nicht nach der Serverzeit, MUSS der Request mit 422 Unprocessable Content abgelehnt werden.

MUST Wird ein aktiver Override durch eine zukünftige geplante Wartung ersetzt, MUSS zunächst seine aktuelle Wirkung aufgehoben und der Grundzustand neu bestimmt werden; anschließend wird der scheduled Override im selben atomaren Vorgang gespeichert. Ändern sich dabei Betriebsstatus oder Statusmetadaten, gilt statusChanged, andernfalls serviceInfoChanged.

MUST action=clear MUSS einen vorhandenen scheduled oder active Override entfernen. Nach einem aktiven Override wird der wirksame Status im selben atomaren Vorgang aus Lifecycle und aktuellem Grundzustand neu bestimmt. action=clear ohne vorhandenen Override ist ein No-op nach Kapitel 5.3.

4.7.3 Zeitgesteuerte Wartung

MUST Bei einem scheduled Override bleiben operationalStatus und seine Metadaten bis zur Aktivierung unverändert. Das Anlegen, Ändern oder Aufheben erzeugt eine Revision mit serviceInfoChanged, sofern nicht durch das Ersetzen eines aktiven Overrides zugleich Statusdaten geändert werden.

MUST Erreicht die Serverzeit maintenanceWindow.start, MUSS der Verzeichnisdienst den Override atomar auf active setzen, activatedAt bilden, operationalStatus=maintenance und operationalStatusSource=manualOverride setzen und genau eine neue Revision mit statusChanged erzeugen.

MUST Mit maintenanceWindow.end beziehungsweise validUntil MUSS der Override automatisch enden. Der Verzeichnisdienst entfernt manualOverride, bewertet den Betriebsstatus nach Lifecycle und aktuellem Grundzustand neu und erzeugt genau eine Revision mit statusChanged.

MUST Ein nicht auf maintenance gerichteter Override wird unmittelbar active. validUntil ist dabei optional; ist es vorhanden, endet der Override zu diesem Zeitpunkt automatisch.

MUST Für eine Wartung MÜSSEN validUntil und maintenanceWindow.end identisch sein. maintenanceWindow.start MUSS vor maintenanceWindow.end liegen.

4.8 Initialzustand, Erweiterbarkeit und Größe

MUST Bei Anlage und Wiederanlage eines nicht-retired Eintrags MUSS der Verzeichnisdienst operationalStatus=down, operationalStatusSource=initialRegistration, operationalStatusReason=initialRegistration, eine erklärende operationalStatusDescription und operationalStatusChangedAt=Commit-Zeitpunkt setzen. Dies gilt auch bei availabilityMonitoring.enabled=true; erst eine bestätigte availabilityStart-Nachricht oder ein aktiver manueller Override darf den Status ändern. Bei lifecycleStatus=retired gelten stattdessen die Regeln lifecycleRetired/down gemäß Kapitel 4.5.

Der vollständige DirectoryRecord und seine serverseitigen Unterobjekte sind ausdrücklich erweiterbar. Clients müssen unbekannte optionale Response-Eigenschaften derselben Major-Version ignorieren. DirectoryRecordWrite, ManualOverrideRequest und die WebSocket-Client-Nachrichten sind geschlossen.

MUST NOT Der UTF-8-serialisierte vollständige DirectoryRecord DARF 921.600 Byte nicht überschreiten. Die Grenze MUSS vor dem Commit jeder Anlage oder Änderung geprüft werden.

5 Synchrone Verzeichnisdienst-API

5.1 Ressource und Operationen

`/v1/records/{providerId}/{apiId}/{majorVersion}`

Methode	H2-Business-Process	Vorbedingung	Erfolg
GET	directoryRecordQuery	If-None-Match optional	200 DirectoryRecord + ETag; 304 ohne Body bei unverändertem ETag
PUT Anlage	directoryRecordMaintenance	If-None-Match: *	201 DirectoryRecord + ETag
PUT Änderung	directoryRecordMaintenance	If-Match: "rev-n"	200 DirectoryRecord + ETag
PATCH	directoryRecordMaintenance	If-Match: "rev-n"	200 DirectoryRecord + ETag
DELETE	directoryRecordMaintenance	If-Match: "rev-n"	200 DirectoryRecordDeletionResult

Die vollständige maschinenlesbare Beschreibung steht in openapi-directory-service-v1.yaml. Für Requests mit Body ist application/json zu verwenden. GET und DELETE haben keinen Body; H2-Digest enthält dann den SHA-256-Wert des leeren Oktettstroms.

MUST Die Werte directoryRecordQuery, directoryRecordMaintenance und directoryWebSocket MÜSSEN durch die API-Governance als zulässige H2-Business-Process-Werte registriert sein.

5.2 PUT-Objekt

DirectoryRecordWrite enthält ausschließlich allowedRoles, communicationParameters, capabilities, contacts, validity und availabilityMonitoring. Schlüssel-, Governance- und Systemfelder sind unzulässig.

MUST Bei einer Anlage MUSS der Verzeichnisdienst serviceName, serviceDescription, lifecycleStatus und gegebenenfalls weitere Governance-Felder aus dem API-Katalog der aktuellen Umgebung übernehmen. Bei einer Änderung bleiben die aktuellen Governance-Felder maßgeblich.

MUST apiId und majorVersion MÜSSEN im API-Katalog der Umgebung freigegeben sein. allowedRoles, capabilities und supportedHttpMethods MÜSSEN zulässige Teilmengen der Katalog- beziehungsweise API-Spezifikation sein; Verstöße werden mit 422 Unprocessable Content abgelehnt.

MUST Bei PUT MUSS genau eine der Vorbedingungen If-None-Match: * oder If-Match: "rev-n" vorhanden sein. Fehlen beide Header, gilt 428 Precondition Required; sind beide vorhanden oder

syntaktisch ungültig, gilt 400 Bad Request; ist die gültige Vorbedingung nicht erfüllt, gilt 412 Precondition Failed.

5.3 PATCH-Objekt

PATCH ist in Version 1 ausschließlich für ManualOverrideRequest zugelassen. action=set setzt beziehungsweise ersetzt den Override. action=clear hebt einen geplanten oder aktiven Override auf. Ein generisches Merge-Patch anderer Eigenschaften ist nicht vorgesehen.

MUST action=clear ohne vorhandenen Override MUSS 200 OK mit unverändertem DirectoryRecord und unverändertem ETag liefern. Revision, fachliches Zustandsaudit, Outbox-Ereignis und WebSocket-Benachrichtigung dürfen nicht erzeugt werden. Der technische Request und sein Idempotenzresultat MÜSSEN dennoch protokolliert werden.

5.4 Löschung

Eine erfolgreiche Löschung entfernt den Eintrag aus dem operativen Verzeichnis, erhält die Revisions-Hochwassermarke und liefert 200 OK mit DirectoryRecordDeletionResult. Die WebSocket-Benachrichtigung entryDeleted verwendet dieselbe Revision und entry=null. Die Subscription endet mit dieser Benachrichtigung.

MUST Bestehen fachliche, regulatorische, technische oder revisionsbezogene Gründe gegen eine Löschung, MUSS 409 Conflict mit fachlicher ErrorResponse zurückgegeben werden. Eine Deaktivierung ist dann über Lifecycle- oder Betriebsstatus abzubilden.

5.5 Signatur- und Response-Regeln

Response	Body	Signatur
200/201 DirectoryRecord	ja	fachlich relevant; JWS-signiert; ETag
200 DirectoryRecordDeletion-Result	ja	fachlich relevant; JWS-signiert
304 Not Modified	nein	nicht signiert; ETag und H2-API-Version
409 DELETION_CONFLICT	Error-Response	fachlich; JWS-signiert
409 IDEMPOTENCY_CONFLICT	Error-Response	technisch; nicht signiert
422	Error-Response	fachlich relevant; JWS-signiert
400, 401, 403, 404, 412, 415, 428, 429, 5xx	Error-Response, soweit korrelierbar	technisch; nicht signiert

Alle fachlichen HTTP-Requests sind gemäß [JWS-SPEC] signaturpflichtig. Bei nicht vertrauenswürdig bestimmbarer oder syntaktisch ungültiger H2-Transaction-Id gelten die bodylosen Ausnahmen der API-Guideline.

5.6 ETag, Retry, Idempotenz und atomare Verarbeitung

MUST Ein Retry MUSS dieselbe fachliche Operation einschließlich If-Match beziehungsweise If-None-Match enthalten. Eine Änderung der Vorbedingung ist ein neuer fachlicher Aufruf mit neuer H2-Transaction-Id ohne H2-Initial-Transaction-Id.

MUST Bei einem erkannten Retry MUSS die Idempotenzprüfung vor einer erneuten ETag-Prüfung erfolgen. Ein gespeichertes Ergebnis wird ohne erneute Zustandsänderung, Revision oder Outbox-Erzeugung zurückgegeben.

Der Idempotenzschlüssel besteht aus H2-Message-Sender und effektiver Transaktions-ID. Der gespeicherte Fingerprint umfasst mindestens H2-Message-Receiver, H2-Business-Process, Methode, normalisierte URI, Body-Hash sowie Vorhandensein und Wert von If-Match oder If-None-Match. Die Aufbewahrungsdauer beträgt 24 Stunden.

MUST Ein PUT, dessen marktpartnerseitige Daten vollständig dem aktuellen Stand entsprechen, MUSS 200 OK mit unverändertem DirectoryRecord und ETag liefern. Es entstehen keine Revision, kein fachliches Zustandsaudit, kein Outbox-Ereignis und keine WebSocket-Benachrichtigung. Der technische Request und sein Idempotenzresultat MÜSSEN dennoch protokolliert werden.

Bei einer tatsächlichen Änderung bilden Idempotenzreservierung, Vorbedingungsprüfung, Berechtigungs- und Signaturprüfung, Zustandsänderung, Revision, Audit, gespeichertes Ergebnis und Transactional Outbox eine atomare Verarbeitung. Sichtbarkeit und Benachrichtigung erfolgen erst nach Commit.

5.7 Betriebs- und Retryprofil

Parameter	Wert
Client-Timeout	30 Sekunden
Versuche	höchstens 5 insgesamt: 1 Erstversuch + 4 Retries
Backoff	1, 2, 4, 8 Sekunden; zusätzlich 0–50 % Jitter
Retry-After	Ein längerer gültiger Wert hat Vorrang
Automatischer Retry	Transportfehler, fehlende/unverwertbare Response, 429, 500, 502, 503, 504
Kein automatischer Retry	sonstige 4xx-Responses
Idempotenzaufbe-wahrung	24 Stunden
Gleichzeitige Re-quests	höchstens 20 je MP-ID und Umgebung
Requestrate	höchstens 600 je 60 Sekunden und MP-ID/Umgebung
Limitreaktion	429; bei systemweiter Überlastung 503

6 Zustandsänderungen und WebSocket-Ereignisse

Änderung	Revision	eventType
Anlage eines Eintrags	+1	entryUpserted
Unveränderter PUT	keine	keine Nachricht
Lifecycle oder wirksame Betriebsstatusdaten geändert	+1	statusChanged
Zieladresse, Protokoll, Methoden, Kontakte, Serviceinformationen oder Monitoringkonfiguration/-status geändert	+1	serviceInfoChanged
allowedRoles, capabilities, Gültigkeit oder sonstige Record-Daten geändert	+1	entryUpserted
Geplante Wartung angelegt/geändert/gelöscht, noch nicht aktiv	+1	serviceInfoChanged, sofern kein Status betroffen ist
Geplante Wartung wird aktiv	+1	statusChanged
Aktiver Override endet oder wird aufgehoben	+1	statusChanged
Nur availabilityMonitoringStatus ändert sich	+1	serviceInfoChanged
Monitoringstatus und operationalStatus im selben Vorgang	+1	statusChanged
Heartbeat verlängert confirmed	keine	keine Nachricht
Löschung	+1	entryDeleted

Erfüllt eine Änderung mehrere Kategorien, gilt die Priorität entryDeleted vor statusChanged vor serviceInfoChanged vor entryUpserted. Für jeden abgeschlossenen Änderungsvorgang wird je betroffener aktiver Subscription genau eine notification erzeugt.

7 WebSocket-Integration und externe Schemata

7.1 Schemaauslagerung

Für die vorgesehene Auslagerung werden die vollständigen JSON-Schema-Blöcke der Kapitel 11.3 und 11.4 von [WS-SPEC] durch die folgenden externen Dateien dieses Releases ersetzt. Nach Umsetzung der Änderungshinweise bleiben die Kapitelüberschriften und normativen Verweise bestehen; nur die eingebetteten Schema-Blöcke entfallen:

- ws-client-message.schema.yaml – urn:h2:directory:ws:client-message:v1
- ws-server-message.schema.yaml – urn:h2:directory:ws:server-message:v1
- directory-record.schema.yaml – urn:h2:directory:record:v1

`notification.payload.entry -> urn:h2:directory:record:v1#/$defs/DirectoryRecord`

MUST Das Client-Schema MUSS auf Envelope- und Payload-Ebene geschlossen bleiben. Das Server-Schema MUSS zusätzliche optionale Servereigenschaften und unbekannte künftige Server-Nachrichtentypen tolerieren und signature in allen Server-Nachrichten verbieten.

7.2 AsyncAPI-Vertrag

asyncapi-directory-websocket-v1.yaml beschreibt den Kanal /v1/ws/subscriptions, die Umgebungen PROD und TEST, das Subprotokoll h2.directory.v1, die Upgrade-Header, die signaturpflichtigen Client-Nachrichten, die unsignierten Server-Nachrichten, Close- und Fehlercodes sowie alle Betriebsparameter aus [WS-SPEC].

Die Dokumentperspektive ist der Verzeichnisdienst: receiveClientMessages beschreibt Client→Server; sendServerMessages beschreibt Server→Client. Die Zustellung von notifications bleibt at-most-once ohne Resume oder Anwendungs-Acknowledgement.

8 Serverseitige Semantik außerhalb von JSON Schema

Folgende Prüfungen sind verbindlich, obwohl sie nicht vollständig mit Standard-JSON-Schema ausdrückbar sind:

- providerId, apiId und majorVersion stimmen mit Pfad, Authentisierung, Berechtigung, Umgebung und API-Katalog überein.
- allowedRoles und capabilities sind Teilmengen der für apiId und majorVersion zugelassenen Katalogwerte.
- targetUrl besitzt einen DNS-Namen und keine Benutzerinformation oder IP-Literale, ist für die konkrete API zugelassen und bezeichnet einen Endpunkt derselben Umgebung.
- validity.validUntil liegt nach validity.validFrom.
- deprecatedSince liegt vor sunsetAt; bei erstmaliger Setzung liegt sunsetAt in der Zukunft.
- maintenanceWindow.start liegt vor maintenanceWindow.end; validUntil entspricht maintenanceWindow.end.
- Ein scheduled Override ist ausschließlich maintenance und liegt bei Annahme in der Zukunft; ein abgelaufenes Fenster wird abgelehnt.
- Bei active stimmen die wirksamen operationalStatus-Metadaten mit manualOverride überein; operationalStatusChangedAt entspricht activatedAt. degraded und maintenance besitzen stets einen aktiven Override.
- Bei scheduled bleibt operationalStatusSource ungleich manualOverride und der aktuelle Status unverändert; beim Ersatz eines aktiven Overrides wird zuvor der Grundzustand wirksam.
- Revision, ETag, Auditdatensatz, Idempotenzresultat und Outbox-Ereignis werden konsistent und atomar fortgeschrieben.
- Der UTF-8-serialisierte DirectoryRecord überschreitet 921.600 Byte nicht.
- Monitoringstatus, Karenzzeiten und Circuit-Breaker werden nach [WS-SPEC] ausgewertet.

9 Beispiele

9.1 DirectoryRecordWrite

```
{
  "allowedRoles": ["GridOperator"],
  "communicationParameters": {
    "targetUrl": "https://api.example.de/v1/preliminaryMeasurements",
    "protocol": "https",
    "supportedHttpMethods": ["POST"]
  },
  "capabilities": [],
  "contacts": {
    "technical": {
      "name": "API-Betrieb",
      "organisation": "Beispiel Netz GmbH",
      "email": "api-betrieb@example.de",
      "phone": "+49 30 123456"
    }
  },
  "validity": {"validFrom": "2028-01-01T00:00:00Z"},
  "availabilityMonitoring": {
    "enabled": true,
    "mechanism": "websocketHeartbeat"
  }
}
```

9.2 Geplante Wartung setzen

```
{
  "action": "set",
  "operationalStatus": "maintenance",
  "reason": "plannedMaintenance",
  "description": "Geplante Wartung des Zielsystems.",
  "reference": "CHG-2028-0315",
  "validUntil": "2028-03-15T04:00:00Z",
  "maintenanceWindow": {
    "start": "2028-03-15T02:00:00Z",
    "end": "2028-03-15T04:00:00Z"
  }
}
```

Vor dem Start enthält der Response-Record manualOverride.state=scheduled, während der aktuelle operationalStatus unverändert bleibt. Ab dem Start enthält der Record state=active, activatedAt und operationalStatus=maintenance.

9.3 Vollständiger DirectoryRecord während aktiver Wartung


```
{
  "providerId": "9910567800001",
  "apiId": "H2API00001",
  "majorVersion": 1,
  "revision": 19,
  "serviceName": "Übermittlung vorläufiger Messwerte",
  "serviceDescription": "Übermittlung vorläufiger Messwerte an den Data
Hub.",
  "allowedRoles": ["GridOperator"],
  "communicationParameters": {
    "targetUrl": "https://api.example.de/v1/preliminaryMeasurements",
    "protocol": "https",
    "supportedHttpMethods": ["POST"]
  },
  "capabilities": [],
  "contacts": {
    "technical": {
      "name": "API-Betrieb",
      "organisation": "Beispiel Netz GmbH",
      "email": "api-betrieb@example.de",
      "phone": "+49 30 123456"
    }
  },
  "validity": {"validFrom": "2028-01-01T00:00:00Z"},
  "lifecycleStatus": "active",
  "operationalStatus": "maintenance",
  "operationalStatusSource": "manualOverride",
  "operationalStatusChangedAt": "2028-03-15T02:00:00Z",
  "operationalStatusReason": "plannedMaintenance",
  "operationalStatusDescription": "Geplante Wartung des Zielsystems.",
  "operationalStatusExpectedEnd": "2028-03-15T04:00:00Z",
  "operationalStatusReference": "CHG-2028-0315",
  "manualOverride": {
    "state": "active",
    "operationalStatus": "maintenance",
    "reason": "plannedMaintenance",
    "description": "Geplante Wartung des Zielsystems.",
    "reference": "CHG-2028-0315",
    "setAt": "2028-03-01T10:00:00Z",
    "setBy": {
      "actorType": "marketPartner",
      "marketPartnerId": "9910567800001",
      "role": "GridOperator"
    },
    "activatedAt": "2028-03-15T02:00:00Z",
    "validUntil": "2028-03-15T04:00:00Z",
    "maintenanceWindow": {
      "start": "2028-03-15T02:00:00Z",
      "end": "2028-03-15T04:00:00Z"
    }
  },
```

```
  "availabilityMonitoring": {
    "enabled": true,
    "mechanism": "websocketHeartbeat"
  },
  "availabilityMonitoringStatus": {
    "state": "confirmed",
    "changedAt": "2028-03-15T01:59:00Z",
    "reason": "availabilityConfirmed"
  }
}
```

9.4 Löschbestätigung

```
{
  "providerId": "9910567800001",
  "apiId": "H2API00001",
  "majorVersion": 1,
  "revision": 20,
  "deletedAt": "2028-04-02T00:00:00Z",
  "status": "deleted"
}
```

10 Konformität und Veröffentlichung

MUST Eine Implementierung ist nur konform, wenn sie die Textspezifikationen und alle maschinenlesbaren Artefakte desselben Releases gemeinsam umsetzt.

Konformitätstests müssen mindestens positive und negative Schemafälle, ETag- und Retry-Reihenfolge, No-op-Verhalten, Revision und Outbox, geplante Wartungsaktivierung, Ablauf und Aufhebung, WebSocket-URN-Auflösung, Nachrichtensignatur, Größenlimits, Rate-Limits sowie Resynchronisation prüfen.

Die mit diesem Dokument veröffentlichte OpenAPI- und AsyncAPI-Fassung kodifiziert die festgelegte Semantik. Spätere Änderungen unterliegen dem Änderungsmanagement der API-Guideline und dürfen innerhalb derselben Major-Version keine neue Pflichtangabe oder inkompatible Bedeutung einführen.

11 Quellen

[API-GL] API-Guideline Wasserstoffmarkt, Version 1.0, Working Document, Stand 11.07.2026.

[JWS-SPEC] JWS-Inhaltsdatensicherung für API-Webdienste, Working Document, gemeinsam veröffentlichte Fassung.

[WS-SPEC] Technische Schnittstellenbeschreibung WebSocket-API des Verzeichnisdienstes, Version 1.0, Working Document, Stand 12.07.2026.

[JSON-SCHEMA-2020-12] JSON Schema Draft 2020-12, Core and Validation Specifications.

[OPENAPI-3.1.2] OpenAPI Specification Version 3.1.2.

[ASYNCAPI-3.0.0] AsyncAPI Specification Version 3.0.0.

[RFC2119] Key words for use in RFCs to Indicate Requirement Levels.

[RFC8174] Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words.

Version	Datum	Änderung	Autor
0.9	28.09.2026	Initiale Erstellung	Arbeitsgruppe API